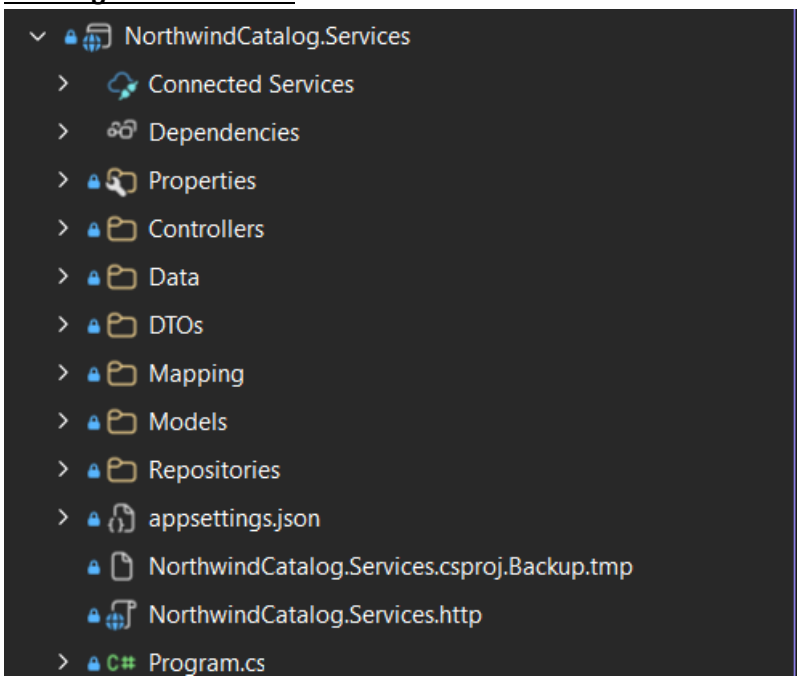
Day 88

1. **Basic WorkFlow of the Project**

   1. User opens browser
   ↓
   2. Request goes to MVC Controller
   ↓
   3. MVC Controller uses HttpClient
   ↓
   4. HttpClient calls API endpoint (URL)
   ↓
   5. API Controller receives request
   ↓
   6. API Controller calls Repository
   ↓
   7. Repository uses DbContext
   ↓
   8. DbContext fetches data from Database
   ↓
   9. Data goes back to Repository
   ↓
   10. Repository returns data to API Controller
   ↓
   11. API Controller maps Entity → DTO (AutoMapper)
   ↓
   12. API returns JSON response
   ↓
   13. MVC Controller receives JSON
   ↓
   14. MVC passes data to Razor View
   ↓
   15. Razor View displays UI

2. **Creating the API Service**



Day 88

3. Adding the Fluent API

NorthwindCatalog.Services | NorthwindCatalog.Services.Data.N | NorthwindContext()

```
using System;
.ng System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using NorthwindCatalog.Services.Models;

namespace NorthwindCatalog.Services.Data;

8 references
public partial class NorthwindContext : DbContext
{
    0 references
    public NorthwindContext()
    {
    }

    0 references
    public NorthwindContext(DbContextOptions<NorthwindContext> options)
        : base(options)
    {
    }

    2 references
    public virtual DbSet<Category> Categories { get; set; }

    1 reference
    public virtual DbSet<Product> Products { get; set; }

    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
#warning To protect potentially sensitive information in your connection string, you should move it out of source c
        => optionsBuilder.UseSqlServer("Server=(localdb)\\MSSQLLocalDB;Database=Northwind;Trusted_Connection=True;"

    0 references
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Category>(entity =>
        {
            entity.HasIndex(e => e.CategoryName, "CategoryName");

            entity.Property(e => e.CategoryId).HasColumnName("CategoryID");
            entity.Property(e => e.CategoryName).HasMaxLength(15);
            entity.Property(e => e.Description).HasColumnType("ntext");
            entity.Property(e => e.Picture).HasColumnType("image");

            entity.Property(e => e.CategoryName).IsRequired();
        });

        modelBuilder.Entity<Product>(entity =>
        {
            entity.HasIndex(e => e.CategoryId, "CategoriesProducts");
```

4. Creating DTOs

- DTOs
  - CategoryDto.cs
  - CategorySummaryDto.cs
  - ProductDto.cs

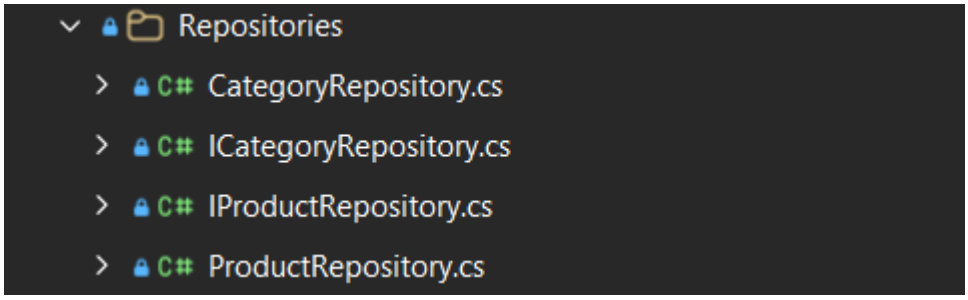
5. Auto Mapper implementation

NorthwindCatalog.Services | NorthwindCatalog.Services.Mappir | MappingProfile()

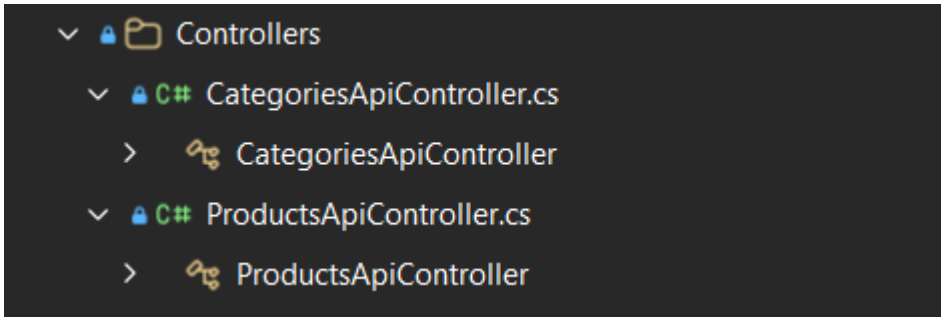
```
using AutoMapper;
using NorthwindCatalog.Services.DTOs;
using NorthwindCatalog.Services.Models; // adjust if needed
namespace NorthwindCatalog.Services.Mapping
{
    2 references
    public class MappingProfile : Profile
    {
        0 references
        public MappingProfile()
        {
            // Category → CategoryDto
            CreateMap<Category, CategoryDto>()
    .ForMember(dest => dest.ImageUrl,
        opt => opt.MapFrom(src =>
            "/images/" + src.CategoryName
                .Replace(" ", "")
                .Replace("/", "") + ".jpeg"));

            // Product → ProductDto
            CreateMap<Product, ProductDto>();
        }
    }
}
```

Day 88

6. **Implementing the Repository Pattern**



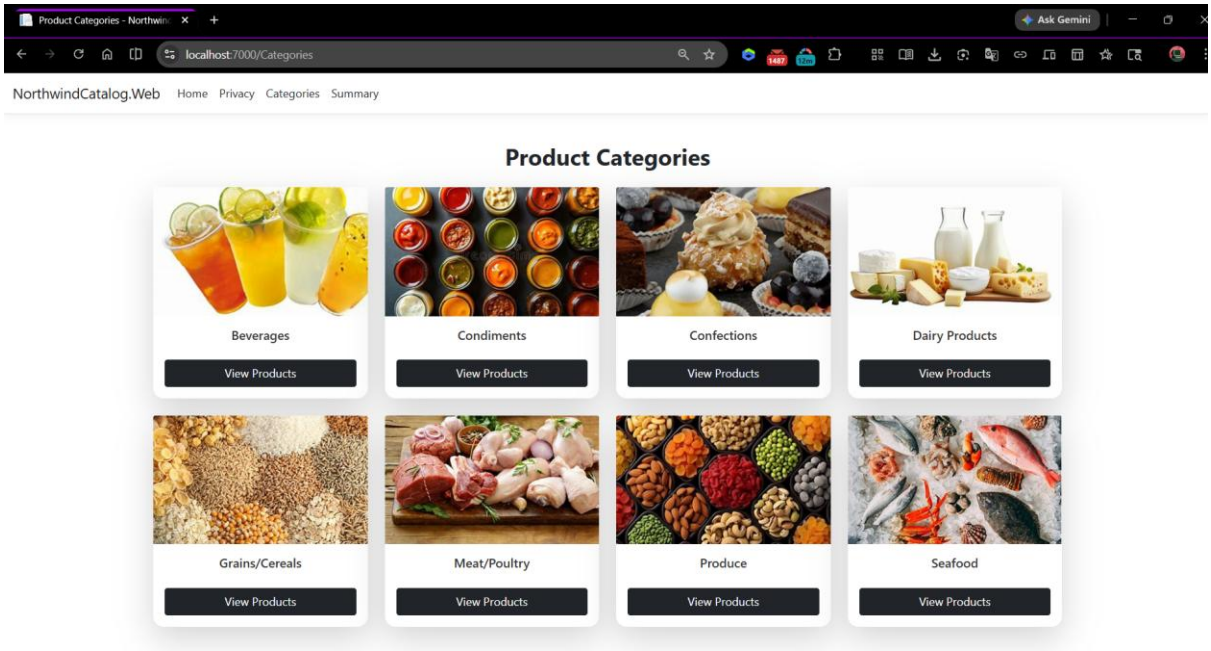
7. **Add the Controllers**



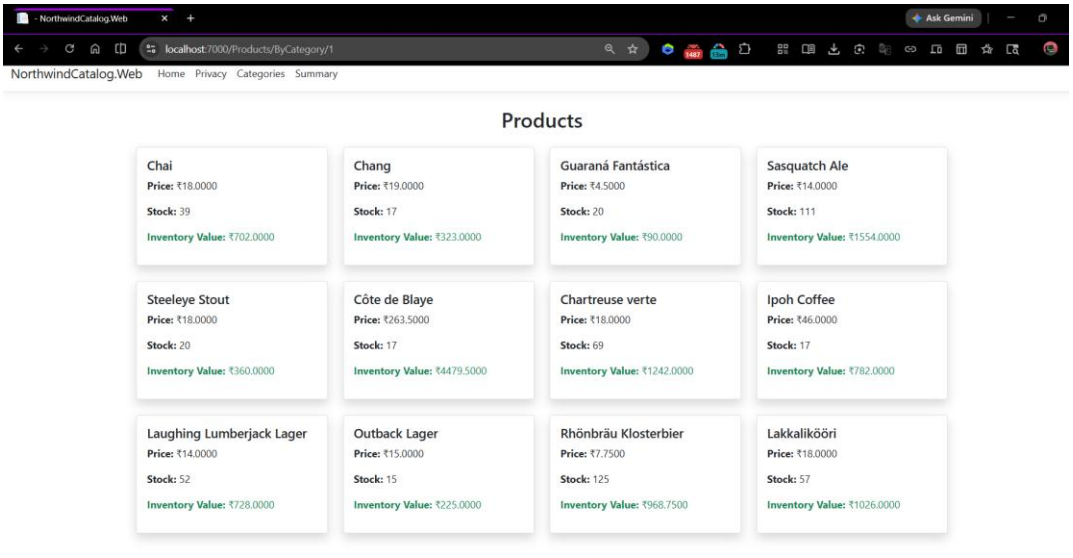
8. **Connecting the MVC with API Service**

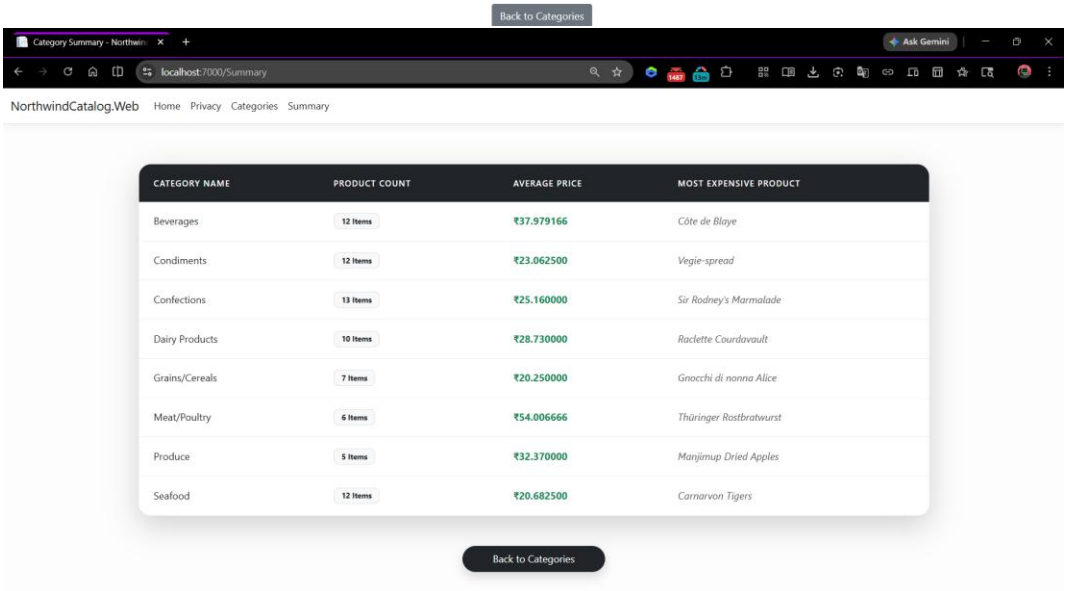
```
builder.Services.AddHttpClient("api", client =>
{
    client.BaseAddress = new Uri("https://localhost:7001/");
});
```

9. **Testing MVC**

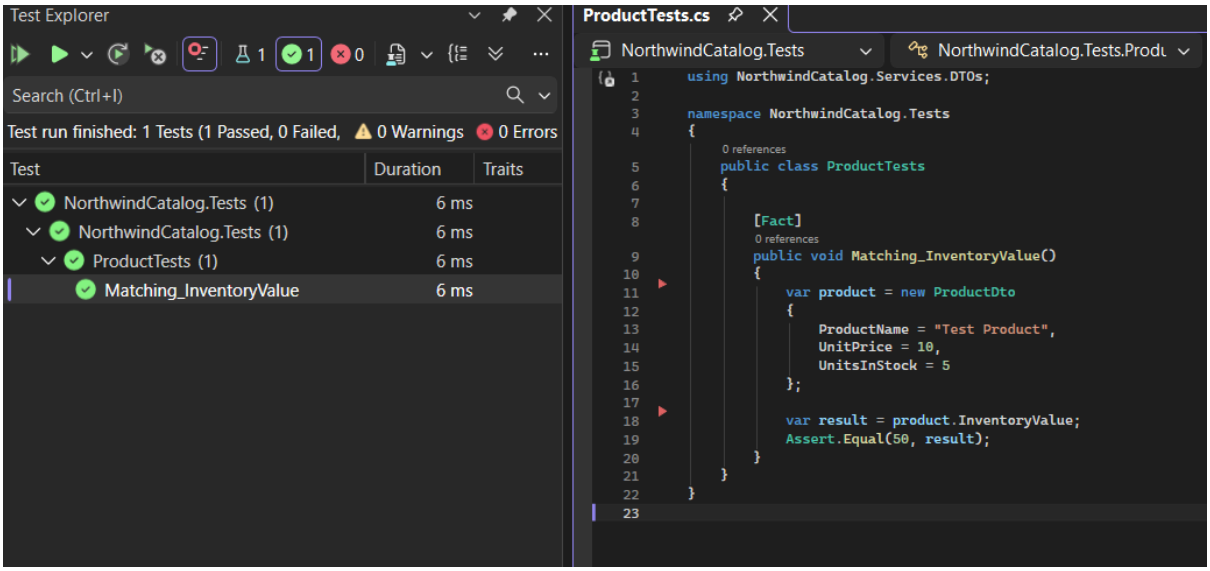


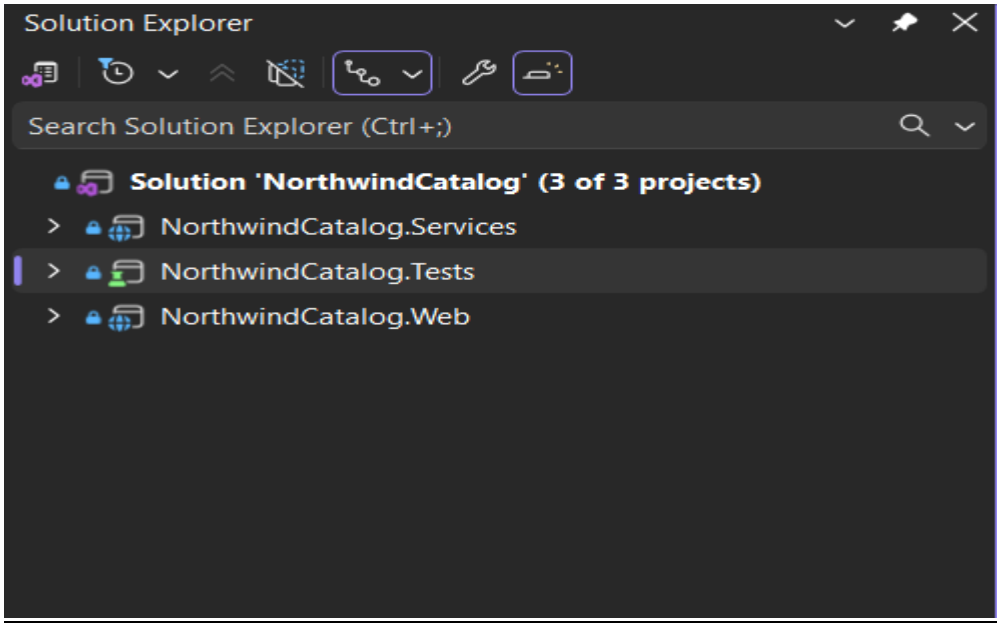
Day 88





10. XUnit Testing



Day 88

11. **Whole Solution Structure**